Understanding Flow Matching from a mathematical view

Pipi Hu

Beijing Institute of Mathematical Sciences and Applications

October 6, 2025

Feynman's words

What I cannot create, I do not understand.

Mathematical motivation for Flow Matching

The fundamental challenge: Learn a velocity field $\mathbf{v}(\mathbf{x}, t)$ that transforms a simple base distribution $p_1(\mathbf{x})$ into the data distribution $p_0(\mathbf{x})$ through the continuity equation:

$$\partial_t p(\mathbf{x}, t) + \nabla \cdot \big(p(\mathbf{x}, t)\, \mathbf{v}(\mathbf{x}, t)\big) = 0. \tag{1}$$

Key advantages over score-based methods:

- **No score estimation**: Avoids computing $\nabla_{\mathbf{x}} \log p(\mathbf{x}, t)$ during training
- **Analytic supervision**: Provides closed-form velocity targets via conditional paths
- **Deterministic sampling**: Enables ODE integration with large solver steps
- **Mathematical elegance**: Direct connection to optimal transport theory

Outline

The continuity equation: mathematical foundation

Theorem: Let $\mathbf{v}(\mathbf{x}, t)$ be a smooth velocity field. Then the density $p(\mathbf{x}, t)$ transported by the ODE $\dot{\mathbf{x}} = \mathbf{v}(\mathbf{x}, t)$ satisfies the continuity equation:

$$\partial_t p(\mathbf{x}, t) + \nabla \cdot \big(p(\mathbf{x}, t)\, \mathbf{v}(\mathbf{x}, t)\big) = 0. \tag{2}$$

Proof sketch: For any test function $\phi(\mathbf{x})$, we have:

$$\frac{d}{dt} \int \phi(\mathbf{x}) p(\mathbf{x}, t) d\mathbf{x} = \int \phi(\mathbf{x}) \partial_t p(\mathbf{x}, t) d\mathbf{x} \tag{3}$$

$$= \int \phi(\mathbf{x}) \mathbf{v}(\mathbf{x}, t) \cdot \nabla p(\mathbf{x}, t) d\mathbf{x} \tag{4}$$

$$= - \int \nabla \phi(\mathbf{x}) \cdot \mathbf{v}(\mathbf{x}, t) p(\mathbf{x}, t) d\mathbf{x} \tag{5}$$

The last equality follows from integration by parts and the chain rule.

Derivation: target velocity for Gaussian path

Given $\mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0) + \sigma_t \boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$,

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbf{x}_t = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t\,\boldsymbol{\epsilon}. \tag{6}$$

Condition on $\mathbf{x}_t = \mathbf{x}$ and $\mathbf{x}_0$. Since $\boldsymbol{\epsilon} = (\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0))/\sigma_t$,

$$\begin{aligned}
\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) &= \mathbb{E}\big[\tfrac{\mathrm{d}}{\mathrm{d}t}\mathbf{x}_t \mid \mathbf{x}_t = \mathbf{x},\, \mathbf{x}_0\big] && (7)\\
&= \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t\,\mathbb{E}[\boldsymbol{\epsilon} \mid \mathbf{x}_t = \mathbf{x},\, \mathbf{x}_0] && (8)\\
&= \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t\,\frac{\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0)}{\sigma_t} && (9)\\
&= \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \frac{\dot{\sigma}_t}{\sigma_t}\Big(\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0)\Big). && (10)
\end{aligned}$$

Conditional probability paths: mathematical framework

Definition: Given a data distribution $p_{\text{data}}(\mathbf{x}_0)$ and base distribution $p_{\text{base}}(\mathbf{x})$, we define conditional paths $q_t(\mathbf{x} \mid \mathbf{x}_0)$ such that:

$$p_t(\mathbf{x}) = \int p_{\text{data}}(\mathbf{x}_0)\, q_t(\mathbf{x} \mid \mathbf{x}_0)\, \mathrm{d}\mathbf{x}_0, \quad t \in [0,1], \tag{11}$$

with boundary conditions $p_0 = p_{\text{data}}$ and $p_1 = p_{\text{base}}$.
Key insight: The conditional velocity field

$$\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) := \mathbb{E}\left[\tfrac{\mathrm{d}}{\mathrm{d}t}\mathbf{x}_t \;\middle|\; \mathbf{x}_t = \mathbf{x},\, \mathbf{x}_0\right] \tag{12}$$

provides *analytic supervision* for learning the marginal velocity field $\mathbf{v}(\mathbf{x}, t)$.
Mathematical advantage: No need to estimate $\nabla_{\mathbf{x}} \log p(\mathbf{x}, t)$ during training!

Outline

Gaussian conditional paths: mathematical derivation

Assumption: Gaussian conditional paths

$$q_t(\mathbf{x} \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x};\ \boldsymbol{\mu}_t(\mathbf{x}_0),\, \sigma_t^2 \mathbf{I}\big), \quad \mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0) + \sigma_t \boldsymbol{\epsilon}. \tag{13}$$

Mathematical derivation: Taking the time derivative of the path:

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbf{x}_t = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t\, \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} = \frac{\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0)}{\sigma_t}. \tag{14}$$

Conditional expectation: Since $\boldsymbol{\epsilon}$ is independent of $\mathbf{x}_t$ given $\mathbf{x}_0$, we have:

$$\begin{aligned}
\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) &= \mathbb{E}\big[\tfrac{\mathrm{d}}{\mathrm{d}t}\mathbf{x}_t \;\big|\; \mathbf{x}_t = \mathbf{x},\, \mathbf{x}_0\big] && (15)\\
&= \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t\, \mathbb{E}[\boldsymbol{\epsilon} \mid \mathbf{x}_t = \mathbf{x},\, \mathbf{x}_0] && (16)\\
&= \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \dot{\sigma}_t\, \frac{\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0)}{\sigma_t} && (17)
\end{aligned}$$

Final result:

$$\boxed{\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + \frac{\dot{\sigma}_t}{\sigma_t}\Big(\mathbf{x} - \boldsymbol{\mu}_t(\mathbf{x}_0)\Big).} \tag{18}$$

Mathematical analysis of path choices

1. **Linear bridge** (optimal transport):
   - $\boldsymbol{\mu}_t(\mathbf{x}_0) = (1-t)\,\mathbf{x}_0$, $\sigma_t = t\,\sigma_{\max}$
   - **Mathematical property**: Minimizes Wasserstein-2 distance
   - **Velocity**: $\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) = \frac{\sigma_{\max}}{t}(\mathbf{x} - \mathbf{x}_0)$
2. **VP-like** (DDPM-inspired):
   - $\boldsymbol{\mu}_t(\mathbf{x}_0) = \alpha(t)\,\mathbf{x}_0$, $\sigma_t = \sqrt{1-\alpha^2(t)}$
   - **Mathematical property**: Preserves energy structure
   - **Velocity**: $\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) = \dot{\alpha}(t)\mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t}(\mathbf{x} - \alpha(t)\mathbf{x}_0)$
3. **VE-like** (SMLD-inspired):
   - $\boldsymbol{\mu}_t(\mathbf{x}_0) = \mathbf{x}_0$, $\sigma_t = \sigma(t)$ increasing
   - **Mathematical property**: Preserves data structure
   - **Velocity**: $\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) = \frac{\dot{\sigma}(t)}{\sigma(t)}(\mathbf{x} - \mathbf{x}_0)$

Key insight: All choices provide *closed-form* $\mathbf{u}_t$ via the boxed formula!

Derivation: VE/VP target forms

VE-like. $\boldsymbol{\mu}_t(\mathbf{x}_0) = \mathbf{x}_0$, $\sigma_t = \sigma(t)$. Then

$$\mathbf{u}_t = 0 + \frac{\dot{\sigma}_t}{\sigma_t}\,(\mathbf{x} - \mathbf{x}_0). \tag{19}$$

VP-like. $\boldsymbol{\mu}_t(\mathbf{x}_0) = \alpha(t)\,\mathbf{x}_0$, $\sigma_t = \sqrt{1-\alpha^2(t)}$. Using $\mathbf{u}_t = \dot{\boldsymbol{\mu}}_t + (\dot{\sigma}_t/\sigma_t)(\mathbf{x} - \boldsymbol{\mu}_t)$, we get

$$\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) = \dot{\alpha}\,\mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t}\,\mathbf{x} - \frac{\dot{\sigma}_t}{\sigma_t}\,\alpha\,\mathbf{x}_0 \tag{20}$$

$$= \frac{\dot{\sigma}_t}{\sigma_t}\,\mathbf{x} + \left(\dot{\alpha} - \alpha\,\frac{\dot{\sigma}_t}{\sigma_t}\right)\mathbf{x}_0. \tag{21}$$

No explicit form for $\dot{\sigma}_t/\sigma_t$ is required to train; schedules determine it numerically.

Outline

Flow Matching: mathematical training objective

Theoretical foundation: Given conditional paths $q_t(\mathbf{x} \mid \mathbf{x}_0)$, the optimal velocity field satisfies:

$$\mathbf{v}^*(\mathbf{x}, t) = \mathbb{E}_{\mathbf{x}_0 | \mathbf{x}_t = \mathbf{x}} \big[\mathbf{u}_t(\mathbf{x} \mid \mathbf{x}_0) \big] \tag{22}$$

Training procedure:

1. Sample $t \sim \mathcal{U}[0, 1]$, $\mathbf{x}_0 \sim p_{\text{data}}$, $\mathbf{x}_t \sim q_t(\cdot \mid \mathbf{x}_0)$
2. Compute analytic target $\mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0)$ from chosen path
3. Minimize regression loss

Mathematical objective:

$$J_{\text{FM}} = \mathbb{E} \big[\| \mathbf{v}_\theta(\mathbf{x}_t, t) - \mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0) \|^2 \big]. \tag{23}$$

Key mathematical advantage: No score estimation needed; all targets are analytic!

Mathematical optimality of Flow Matching

Theorem: The population minimizer of the Flow Matching objective

$$J_{\mathrm{FM}}(\theta) = \mathbb{E}\big[\|\mathbf{v}_\theta(\mathbf{x}_t, t) - \mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0)\|^2\big] \tag{24}$$

is the posterior-averaged velocity field:

$$\mathbf{v}^*(\mathbf{x}, t) = \mathbb{E}\big[\,\mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0)\,\big|\,\mathbf{x}_t = \mathbf{x},\ t\big]. \tag{25}$$

Mathematical proof:

Proof.

Let $Y = \mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0)$, $X = (\mathbf{x}_t, t)$. For any predictor $g(X)$,

$$\mathbb{E}[\|Y - g(X)\|^2] = \mathbb{E}[\|Y - \mathbb{E}[Y \mid X]\|^2] + \mathbb{E}[\|g(X) - \mathbb{E}[Y \mid X]\|^2] \tag{26}$$

$$+ 2\mathbb{E}[(Y - \mathbb{E}[Y \mid X]) \cdot (g(X) - \mathbb{E}[Y \mid X])] \tag{27}$$

The cross term vanishes by the tower property, and the second term is minimized when $g(X) = \mathbb{E}[Y \mid X]$.

Key insight: Flow Matching learns the *optimal* velocity field that respects the continuity

Mathematical foundation of inference

Theoretical basis: The learned velocity field $\mathbf{v}_\theta(\mathbf{x}, t)$ defines a deterministic ODE:

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t), \quad t \in [0, 1] \tag{28}$$

Mathematical properties:

- **Existence**: Under Lipschitz conditions, solutions exist and are unique
- **Consistency**: The ODE preserves the continuity equation
- **Stability**: Deterministic integration avoids accumulation of stochastic errors

Generation process:

1. Initialize $\mathbf{x}(1) \sim p_1$ (e.g., $\mathcal{N}(0, \mathbf{I})$)
2. Integrate ODE backwards: $\frac{\mathrm{d}}{\mathrm{d}t}\mathbf{x} = \mathbf{v}_\theta(\mathbf{x}, t)$, $t : 1 \to 0$
3. Use standard ODE solvers (Euler, RK4, Dormand-Prince, etc.)

Mathematical advantages:

- Deterministic sampling (reproducible)
- Fast with large solver steps
- No stochastic correctors needed

Outline

Mathematical connection to diffusion models

Theoretical link: For a forward SDE $\mathrm{d}\mathbf{x} = f(\mathbf{x}, t)\,\mathrm{d}t + g(t)\,\mathrm{d}B_t$, the probability flow ODE is:

$$\frac{\mathrm{d}}{\mathrm{d}t}\mathbf{x} = f(\mathbf{x}, t) - \tfrac{1}{2}g^2(t)\,\nabla_{\mathbf{x}} \log p(\mathbf{x}, t). \tag{29}$$

Key mathematical insight: When Flow Matching uses Gaussian paths that match the SDE's marginal distributions p_t, the learned velocity field $\mathbf{v}_\theta$ approximates the probability flow drift:

$$\mathbf{v}_\theta(\mathbf{x}, t) \approx f(\mathbf{x}, t) - \tfrac{1}{2}g^2(t)\,\nabla_{\mathbf{x}} \log p(\mathbf{x}, t) \tag{30}$$

Crucial advantage: Flow Matching achieves this approximation *without* ever computing the score $\nabla_{\mathbf{x}} \log p(\mathbf{x}, t)$ during training!

Mathematical derivation: probability flow ODE

Setup: Forward SDE $\mathrm{d}\mathbf{x} = f(\mathbf{x}, t)\,\mathrm{d}t + g(t)\,\mathrm{d}B_t$ with density $p(\mathbf{x}, t)$ solving the Fokker-Planck equation:

$$\partial_t p = -\nabla \cdot (fp) + \tfrac{1}{2} g^2 \triangle p. \tag{31}$$

Key insight: A deterministic ODE $\dot{\mathbf{x}} = \mathbf{v}(\mathbf{x}, t)$ transports densities via the continuity equation:

$$\partial_t p + \nabla \cdot (p\,\mathbf{v}) = 0. \tag{32}$$

Mathematical derivation: Equating the two equations:

$$-\nabla \cdot (p\,\mathbf{v}) = -\nabla \cdot (fp) + \tfrac{1}{2} g^2 \triangle p \tag{33}$$

$$\Rightarrow\ \mathbf{v} = f - \tfrac{1}{2} g^2 \frac{\nabla p}{p} \ = \ f - \tfrac{1}{2} g^2 \nabla \log p. \tag{34}$$

Result: The probability flow ODE drift is $\mathbf{v} = f - \frac{1}{2} g^2 \nabla \log p$.

Specializing VE/VP-style paths

- VE-style: $\boldsymbol{\mu}_t(\mathbf{x}_0) = \mathbf{x}_0$. Target becomes $\mathbf{u}_t = \frac{\dot{\sigma}_t}{\sigma_t}(\mathbf{x} - \mathbf{x}_0)$.
- VP-style: $\boldsymbol{\mu}_t = \alpha(t)\mathbf{x}_0$, $\sigma_t = \sqrt{1 - \alpha^2(t)}$. Target is $\mathbf{u}_t = \dot{\alpha}\,\mathbf{x}_0 + \frac{\dot{\sigma}_t}{\sigma_t}(\mathbf{x} - \alpha\mathbf{x}_0)$.

Averaging over $\mathbf{x}_0$ under the path recovers the probability flow drift forms used by diffusion models.

Outline

Complete training recipe

Step-by-step procedure:

1. **Sample data**: $\mathbf{x}_0 \sim p_{\text{data}}$, $t \sim \mathcal{U}[0,1]$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$
2. **Forward pass**: $\mathbf{x}_t = \boldsymbol{\mu}_t(\mathbf{x}_0) + \sigma_t \boldsymbol{\epsilon}$
3. **Compute target**: $\mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0) = \dot{\boldsymbol{\mu}}_t(\mathbf{x}_0) + (\dot{\sigma}_t/\sigma_t)(\mathbf{x}_t - \boldsymbol{\mu}_t(\mathbf{x}_0))$
4. **Regression loss**: $\|\mathbf{v}_\theta(\mathbf{x}_t, t) - \mathbf{u}_t(\mathbf{x}_t \mid \mathbf{x}_0)\|^2$

Implementation notes:

- Use standard optimizers (Adam, AdamW)
- Typical learning rates: 10^{-4} to 10^{-3}
- Training time similar to diffusion models

Complete inference recipe

Generation procedure:

1. **Initialize**: $\mathbf{x}(1) \sim p_1$ (e.g., $\mathcal{N}(0, \mathbf{I})$)
2. **ODE integration**: $\dot{\mathbf{x}} = \mathbf{v}_\theta(\mathbf{x}, t)$ from $t = 1$ to $t = 0$
3. **Optional**: Add predictor-corrector steps for higher quality

Solver recommendations:

- **Fast**: Euler method (1-10 steps)
- **Accurate**: RK4, Dormand-Prince (10-50 steps)
- **Adaptive**: Use error control for automatic step sizing

Outline

Motivation

- Goal: map a simple prior (e.g., $\epsilon \sim \mathcal{N}(0, I)$) to the data distribution p_{data}.
- Diffusion / Flow Matching usually require *multi-step* ODE/SDE integration during sampling.
- Desire: **single-step (1-NFE)** generation while retaining high fidelity and stability.
- Key idea in MeanFlow: learn a *ground-truth average velocity field* that summarizes an entire time interval.

Rectified / Flow Matching setup

Given schedules a_t, b_t and time $t \in [0, 1]$:

$$\mathbf{z}_t = a_t\,\mathbf{x} + b_t\,\epsilon, \tag{35}$$

$$\mathbf{v}_t \equiv \frac{d\mathbf{z}_t}{dt} = a'_t\,\mathbf{x} + b'_t\,\epsilon. \tag{36}$$

Marginal velocity (ground-truth field for FM):

$$\mathbf{v}(\mathbf{z}_t, t) \;=\; \mathbb{E}_{p_t(\mathbf{v}_t|\mathbf{z}_t)}\big[\mathbf{v}_t\big]. \tag{37}$$

Training (conditional FM):

$$\mathcal{L}_{\mathrm{CFM}} = \mathbb{E}_{t,\mathbf{x},\epsilon}\left\|v_\theta(\mathbf{z}_t, t) - \mathbf{v}_t\right\|_2^2, \quad \text{with } \mathbf{z}_t = a_t\mathbf{x} + b_t\epsilon. \tag{38}$$

Sampling in Flow Matching

The probability-flow ODE:

$$\frac{d\mathbf{z}_t}{dt} = \mathbf{v}(\mathbf{z}_t, t), \qquad (\text{start from } \mathbf{z}_1 = \epsilon). \tag{39}$$

With a discrete solver (Euler): $\mathbf{z}_{t_{i+1}} \approx \mathbf{z}_{t_i} + (t_{i+1} - t_i)\,\mathbf{v}(\mathbf{z}_{t_i}, t_i)$.

Issue

Coarse discretization along *curved* trajectories leads to significant numerical error $\Rightarrow$ many steps are needed.

Definition: Average Velocity

For any pair $r < t$ and point $\mathbf{z}_t$ along the flow path:

$$\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \;:=\; \frac{1}{t-r}\int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau)\,\mathrm{d}\tau. \tag{40}$$

- $\overline{\mathbf{u}}$ is an **underlying field** induced by $\mathbf{v}$ (no networks yet).
- As $r \to t$, $\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \to \mathbf{v}(\mathbf{z}_t, t)$.
- Composition over intervals holds: one big step equals two smaller consecutive steps (additivity of integrals).

MeanFlow Identity (core relation)

Starting from Eq. (40):

$$(t-r)\,\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \;=\; \int_r^t \mathbf{v}(\mathbf{z}_\tau, \tau)\,\mathrm{d}\tau. \tag{41}$$

Differentiate w.r.t. t (treating r as constant) and use the fundamental theorem of calculus:

$$\boxed{\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \;=\; \mathbf{v}(\mathbf{z}_t, t) \;-\; (t-r)\,\frac{d}{dt}\,\overline{\mathbf{u}}(\mathbf{z}_t, r, t)} \tag{42}$$

where the total derivative expands to

$$\frac{d}{dt}\,\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \;=\; \underbrace{\mathbf{v}(\mathbf{z}_t, t)}_{\frac{d\mathbf{z}_t}{dt}} \cdot \partial_{\mathbf{z}}\overline{\mathbf{u}}(\mathbf{z}_t, r, t) \;+\; \partial_t \overline{\mathbf{u}}(\mathbf{z}_t, r, t). \tag{43}$$

Interpretation: Eq. (42) links instantaneous and average velocities *without* introducing networks.

Trainable objective

Parameterize a network $\overline{\mathbf{u}}_\theta(\mathbf{z}, r, t)$. Use the MeanFlow identity to form a target (replace $\mathbf{v}$ by sample-conditional $\mathbf{v}_t$):

$$\text{Target:} \quad \overline{\mathbf{u}}_{\text{tgt}} = \mathbf{v}_t - (t-r)\big(\underbrace{\mathbf{v}_t \cdot \partial_{\mathbf{z}}\overline{\mathbf{u}}_\theta}_{\text{JVP}} + \partial_t \overline{\mathbf{u}}_\theta\big). \tag{44}$$

$$\mathcal{L}(\theta) = \mathbb{E}\left\| \overline{\mathbf{u}}_\theta(\mathbf{z}_t, r, t) - \mathrm{sg}(\overline{\mathbf{u}}_{\text{tgt}}) \right\|_2^2. \tag{45}$$

- $\mathrm{sg}(\cdot)$: stop-gradient; avoids higher-order differentiation.
- JVP (Jacobian-vector product) computed efficiently via autodiff.
- If $r = t$, the correction term vanishes $\Rightarrow$ reduces to standard Flow Matching.

Pseudocode (training)

MeanFlow training algorithm:

1. Sample $t, r \sim \text{LogitNormal}$; set $t \leftarrow \max(r, t)$, $r \leftarrow \min(r, t)$
2. Sample $\mathbf{x} \sim p_{\text{data}}$, $\epsilon \sim \mathcal{N}(0, I)$; set $\mathbf{z}_t = (1-t)\mathbf{x} + t\epsilon$, $\mathbf{v}_t = \epsilon - \mathbf{x}$
3. Compute $\big(\overline{\mathbf{u}}_\theta,\ \frac{d}{dt}\overline{\mathbf{u}}_\theta\big) = \mathrm{JVP}\big(\overline{\mathbf{u}}_\theta(\cdot, r, t),\ (\mathbf{v}_t, 0, 1)\big)$
4. Set $\overline{\mathbf{u}}_{\text{tgt}} \leftarrow \mathbf{v}_t - (t-r)\,\frac{d}{dt}\overline{\mathbf{u}}_\theta$
5. Minimize $\|\overline{\mathbf{u}}_\theta(\mathbf{z}_t, r, t) - \mathrm{sg}(\overline{\mathbf{u}}_{\text{tgt}})\|_2^2$ (optionally with adaptive weights)

Sampling (1 step or few steps)

Replace the time integral by the learned average velocity:

$$\mathbf{z}_r \;=\; \mathbf{z}_t - (t-r)\,\overline{\mathbf{u}}_\theta(\mathbf{z}_t, r, t). \tag{46}$$

1-NFE sampling: take $(r,t) = (0,1)$, $\mathbf{z}_1 = \epsilon$,

$$\boxed{\mathbf{x} \equiv \mathbf{z}_0 = \epsilon - \overline{\mathbf{u}}_\theta(\epsilon, 0, 1)}\,. \tag{47}$$

- Few-step variants are immediate by chaining intervals.

CFG inside the field (still 1-NFE)

Define a CFG velocity field

$$\mathbf{v}_{\text{cfg}}(\mathbf{z}_t, t \mid c) \;=\; \omega\, \mathbf{v}(\mathbf{z}_t, t \mid c) + (1-\omega)\, \mathbf{v}(\mathbf{z}_t, t). \tag{48}$$

Key insight: CFG can be applied directly to the average velocity field:

$$\overline{\mathbf{u}}_{\text{cfg}}(\mathbf{z}_t, r, t \mid c) \;=\; \omega\, \overline{\mathbf{u}}(\mathbf{z}_t, r, t \mid c) + (1-\omega)\, \overline{\mathbf{u}}(\mathbf{z}_t, r, t). \tag{49}$$

This maintains the 1-NFE property while enabling conditional generation.

Outline

Key references and resources

Core papers:

- Lipman et al., *Flow Matching: A unifying perspective of score-based training and generative modeling* (Guide and Code, 2024)
- Song et al., *Score-based generative modeling through stochastic differential equations* (2021)
- Ho et al., *Denoising Diffusion Probabilistic Models* (2020)

Implementation resources:

- Flow Matching Guide and Code (GitHub)
- Open source libraries: diffusers, torchdyn
- Tutorial notebooks and examples

Welcome inputs

Any comments?

Welcome your inputs!